

Rapport Technique du Projet SD2026 : Nuage de Mots d'Actualité

1. Introduction

Ce rapport détaille la conception et l'implémentation de l'application Flask d'analyse de sitemaps d'actualités et de génération de nuages de mots, répondant au mandat SD2026.

2. Conception et Choix Techniques

2.1. Base de données et Schéma MongoDB

MongoDB a été choisi pour sa flexibilité avec des données non structurées ou semi-structurées comme celles issues du web scraping. Le schéma principal est la collection 'articles' où chaque document représente un article d'actualité. Les champs choisis sont :

- **titre** : Le titre de l'article extrait du XML.
- **url_originale** : L'URL unique de l'article, utilisée comme identifiant fonctionnel.
- **date_publication** : Date convertie en objet DateTime pour faciliter les requêtes temporelles.
- **source_id** : L'identifiant de la source d'actualité (ex: 'lemonde').
- **horodatage_consultation** : Date d'insertion/mise à jour dans notre système.
- **mots_cles** : Un tableau de chaînes de caractères contenant les mots pertinents après passage dans le pipeline NLP (minuscules, tokenisation, suppression des stopwords).

2.2. Stratégie d'Indexation

Plusieurs index ont été mis en place pour optimiser de manière critique les performances :

1. **Index unique sur 'url_originale'** : Cet index garantit qu'il n'y ait aucun doublon lors des multiples passes du scraper (upsert).
2. **Index descendant sur 'date_publication'** : Crucial pour le filtrage par intervalle de temps, car les utilisateurs recherchent généralement les actualités récentes (ex: les 7 derniers jours).
3. **Index ascendant sur 'source_id'** : Optimise les recherches spécifiques à une source.

Justification sur l'indexation des mots-clés : Un index de type 'Text' MongoDB est idéal pour la recherche floue ou sémantique. Cependant, notre cas d'usage nécessite de calculer des fréquences exactes sur des tokens déjà normalisés et nettoyés en amont par NLP. Par conséquent, l'utilisation du pipeline `\$match` sur le champ `mots\_cles` (qui est un simple tableau de chaînes de caractères) est suffisante et plus légère pour des recherches exactes après pré-traitement, évitant ainsi la surcharge d'un index textuel global.

2.3. Pipeline ETL (Ingestion Automatisée)

L'ingestion se fait de manière asynchrone grâce à APScheduler. Le script télécharge le sitemap, extrait les balises XML à l'aide de BeautifulSoup, puis applique directement la transformation NLP (NLTK) sur les titres. Stocker les mots-clés pré-calculés dans le tableau 'mots_cles' au moment de l'insertion permet un gain de performance massif lors des requêtes d'agrégation, évitant de retraiter le texte à chaque demande de l'utilisateur.

2.4. Logique Analytique et Pipeline d'Agrégation

Pour compter la fréquence des mots, plutôt que de rapatrier tous les articles en mémoire Python, l'application délègue le travail à la base de données via le pipeline d'agrégation de MongoDB :

- **\$match** : Filtre les articles par source et/ou date.
- **\$unwind** : Déconstruit le tableau 'mots_cles' pour avoir un document par mot.
- **\$group** : Compte les occurrences de chaque mot (\$sum: 1).
- **\$sort** : Trie les résultats de manière décroissante.

Ce choix déplace la charge sur le serveur de base de données, qui est hautement optimisé pour ce type d'opération.